

Actividad. Análisis de Activación Diferencial - Hipathia

Módulo 4

Javier Luque Serrano

2026-02-15

Contents

Introducción	1
Análisis de Activación Diferencial	2
Obtención de la matriz de expresión normalizada	2
Limpieza de los datos	2
Normalización	3
Exploración de los datos obtenidos	6
Implementación del método Hipathia	7
Exploración y control de calidad	8
Preprocesado para Hipathia: escalado y normalizado	9
Cálculo de activación de circuitos	11
Análisis de activación diferencial: Positive vs Negative	12
Informe interactivo	17
Bibliografía	19
Declaración de uso de IA	19

Introducción

En esta actividad se va a hacer un análisis de activación diferencial, de manera que se pueda entender cómo la transducción de señales se va a propagar de manera diferencial en cada muestra dependiendo de los genes que tenga. En otras palabras, se va a hacer un análisis mecanístico con **hiPathia** (*High Throughput Pathway Interpretation and Analysis*)[1].

Este método está basado en una **red dirigida** (con signos) de interacciones entre genes, de manera que permite la integración de datos de expresión de genes en tejido, proporcionando el contexto celular del cáncer de mama, en este caso. Así, obtenemos un modelo mecanístico mediante la inferencia del contexto celular funcional.

El método utiliza la **expresión de los genes** para simular las **interacciones entre proteínas**, apoyándose de las **relaciones de activación/inhibición** calculadas mediante la propagación de la señal que simula la **transducción de señales** que tiene lugar en la célula, pudiendo así obtener los perfiles de la **actividad funcional diferencial** de las muestras. Para ello se utilizará la base de datos **KEGG** y se hará un breve análisis de expresión diferencial previo.

Así, se descomponen todos los caminos elementales que parten de los distintos nodos de entrada o receptores (inicializados en 1) y acaban en un nodo de salida o efector, el cual lleva una función o gen asociada sobre el cual se va a computar el valor final de la activación, y se combinan para obtener las funciones que pueden tener cada. Así, este método está basado en una propagación recursiva de la señal, de manera que el nivel de

activación es calculada para cada nodo en todos los caminos elementales. Brevemente: *se toman todos los valores de expresión de los genes que componen un nodo y todas las posibles entradas (nodos anteriores), que representan las señales de activación o inhibición de los genes anteriores.*

Esto es similar a un circuito eléctrico donde los nodos son resistencias y las señales de activación, la corriente. Esta “corriente”, que son los valores de transducción de señales, siempre va a estar entre 0 y 1, ya que Hipathia maneja los maneja de manera similar a como si fuesen probabilidades. Así, el reescalado se hará de manera contenida en la información contenida en la expresión de los genes estudiados.

El análisis va a ser similar al de expresión diferencial (misma estadística), pero en lugar de usar la expresión de los genes, tendremos valores de activación, obteniendo al final los genes efectores que están desregulados y las funciones asociadas desreguladas.

Análisis de Activación Diferencial

Este análisis mecanístico sobre las muestras luminales de cáncer de mama nos mostrará la activación diferencial de las diferentes rutas metabólicas que conectan a los genes con determinadas funciones.

Para ello, utilizaremos el paquete de Hipathia en R, utilizando la matriz de expresión que previamente deberemos normalizar (aunque ya se obtuvo en el previo análisis, se va a hacer de forma rápida a modo de práctica).

Obtención de la matriz de expresión normalizada

Limpieza de los datos

Lo primero de todo, siempre, es la obtención de los datasets y la correcta limpieza de los datos.

```
#setwd("D:/Académico/Universitario/Master en Bioinformática/Cursos/
#Módulo 4. Metodología de la Investigación I/Actividades/
#Actividad 4. Análisis de activación diferencial")

DATA_DIR <- '.' #ruta relativa ('.' quiere decir "de aquí en adelante")
OUT_DIR <- file.path(DATA_DIR, 'outputs') # cambiar path
if (!dir.exists(OUT_DIR)) dir.create(OUT_DIR, recursive = TRUE) # crear directorio si no existe

# Cargamos el archivo
exp_mat <- read.delim("./BRCA_exp_matrix.tsv", header=T, sep="\t", fill=T,
                      stringsAsFactors = F)
dim(exp_mat)

## [1] 20500 1155

# Hay valores NA en los nombres de las muestras
clinical <- read.delim("./clinical_info_TCGA-BRCA.tsv", header = TRUE,
                      sep = "\t", check.names = T, fill = TRUE,
                      stringsAsFactors = F, row.names = NULL)

# Quitar valores NA - columna row.names tiene errores
clinical <- clinical[!is.na(clinical$bcr_patient_uid), ]
# Eliminar duplicados mediante una fila que se repite entre muestras
clinical <- clinical[!duplicated(clinical$snames), ]
# Establecer nombre de las filas por orden
rownames(clinical) <- clinical[, 1]
clinical <- clinical[, -1]
rownames(clinical) <- clinical$sample
```

```

cat("Dimensiones (muestras x variables)", dim(clinical))

## Dimensiones (muestras x variables) 1039 114
# Buscamos sólo muestras comunes
sum(colnames(exp_mat)%in%clinical$sample)

## [1] 1030
snames <- intersect(colnames(exp_mat), clinical$sample)

# Escogemos sólo las muestras comunes
exp_mat_clean <- exp_mat[,snames]
clinical_clean <- clinical[snames, ]

# Comprobamos la columna er_status_by_ihc
cat("Grupos de ER:", table(clinical_clean$er_status_by_ihc))

## Grupos de ER: 47 2 224 757
# Eliminamos muestras sin valores en er
clinical_cleaner <- clinical_clean[
  !clinical_clean$er_status_by_ihc=="[Not Evaluated]"
  &!clinical_clean$er_status_by_ihc=="Indeterminate", ]

table(clinical_cleaner$er_status_by_ihc)

##
## Negative Positive
##      224      757
# Nos quedamos con las mismas muestras en la matriz de expresión
snames_clean <- intersect(colnames(exp_mat_clean), clinical_cleaner$sample)
exp_mat_cleaner <- exp_mat_clean[, snames_clean]

# Comprobamos que todas las muestras coinciden en orden
all(rownames(clinical_cleaner)==colnames(exp_mat_cleaner))

## [1] TRUE
# Convertimos a factor la columna de er_status
ER <- factor(clinical_cleaner$er_status_by_ihc, levels = c("Positive", "Negative"))

# Eliminar matrices crudas y renombrar las nuevas
clinical <- clinical_cleaner
exp_mat <- exp_mat_cleaner
rm(clinical_clean,clinical_cleaner, exp_mat_clean, exp_mat_cleaner)

```

Normalización

Una vez tenemos los datos filtrados con las matrices listas para ser usadas, vamos a normalizar con TMM. En este caso, no vamos a tener en cuenta la edad para hacer la matriz de diseño

```

# limma es requerido por edgeR
library(limma)
# Normalización y regresión
library(edgeR)

```

```

# Crear un objeto DGEList que edgeR entienda
y <- DGEList(exp_mat)
class(y)

## [1] "DGEList"
## attr(,"package")
## [1] "edgeR"

str(y)

## Formal class 'DGEList' [package "edgeR"] with 1 slot
## ..@ .Data:List of 2
## .. ..$ : int [1:20500, 1:981] 278 2189 140 607 0 0 347 20 17 1291 ...
## .. ..$- attr(*, "dimnames")=List of 2
## .. .. ..$ : chr [1:20500] "ALPL" "ALPK1" "ALPK2" "ALPK3" ...
## .. .. ..$ : chr [1:981] "SP3930" "SP3918" "SP4007" "SP4019" ...
## .. ..$ : 'data.frame': 981 obs. of 3 variables:
## .. .. ..$ group : Factor w/ 1 level "1": 1 1 1 1 1 1 1 1 1 1 ...
## .. .. ..$ lib.size : num [1:981] 45395506 70692983 63734890 41811807 44575388 ...
## .. .. ..$ norm.factors: num [1:981] 1 1 1 1 1 1 1 1 1 1 ...
## ..$ names: chr [1:2] "counts" "samples"

# Normalizamos en función de la cantidad de lecturas que tenemos
y <- calcNormFactors(y, method = "TMM")
str(y)

## Formal class 'DGEList' [package "edgeR"] with 1 slot
## ..@ .Data:List of 2
## .. ..$ : int [1:20500, 1:981] 278 2189 140 607 0 0 347 20 17 1291 ...
## .. ..$- attr(*, "dimnames")=List of 2
## .. .. ..$ : chr [1:20500] "ALPL" "ALPK1" "ALPK2" "ALPK3" ...
## .. .. ..$ : chr [1:981] "SP3930" "SP3918" "SP4007" "SP4019" ...
## .. ..$ : 'data.frame': 981 obs. of 3 variables:
## .. .. ..$ group : Factor w/ 1 level "1": 1 1 1 1 1 1 1 1 1 1 ...
## .. .. ..$ lib.size : num [1:981] 45395506 70692983 63734890 41811807 44575388 ...
## .. .. ..$ norm.factors: num [1:981] 1.08 0.997 0.985 0.924 0.779 ...
## ..$ names: chr [1:2] "counts" "samples"

# Matriz de diseño para el ajuste de regresión lineal
design <- model.matrix(~ER)

# Quitamos los genes que son estables en la distribución de ambos grupos (caso y control),
# ya que no aportan información
keep <- filterByExpr(y, design)
cat("Filtramos", sum(!keep), "genes") # Nos quitamos 3008 genes

## Filtramos 3695 genes

# Nos quedamos con los genes de keep y todas las muestras,
# manteniendo el tamaño de las librerías
y <- y[keep,,keep.lib.sizes=TRUE]
str(y)

## Formal class 'DGEList' [package "edgeR"] with 1 slot
## ..@ .Data:List of 2
## .. ..$ : int [1:16805, 1:981] 278 2189 140 607 347 20 17 1291 732 1677 ...
## .. ..$- attr(*, "dimnames")=List of 2

```

```
## .. .. ..$ : chr [1:16805] "ALPL" "ALPK1" "ALPK2" "ALPK3" ...
## .. .. ..$ : chr [1:981] "SP3930" "SP3918" "SP4007" "SP4019" ...
## .. ..$ : 'data.frame': 981 obs. of 3 variables:
## .. ..$ group : Factor w/ 1 level "1": 1 1 1 1 1 1 1 1 1 1 ...
## .. ..$ lib.size : num [1:981] 45395506 70692983 63734890 41811807 44575388 ...
## .. ..$ norm.factors: num [1:981] 1.08 0.997 0.985 0.924 0.779 ...
## ..$ names: chr [1:2] "counts" "samples"

# Hacemos una estimacion
y <- estimateCommonDisp(y)

# Ajustamos la distribución al modelo
fit <- glmQLFit(y, design)

# Reproducimos las características que mejor se adaptan al modelo
qlf <- glmQLFTest(fit, coef = "ERNegative")

# Obtenemos las variables que mejor predicen el modelo
pvalues <- topTags(qlf, n=Inf, adjust.method="BH", sort.by = "PValue", p.value=1)
# Convertir en dataframe
p_values <- as.data.frame(pvalues)

# Obtenemos los genes significativos
alpha <- 0.05
sig <- p_values[p_values$FDR<alpha, ]

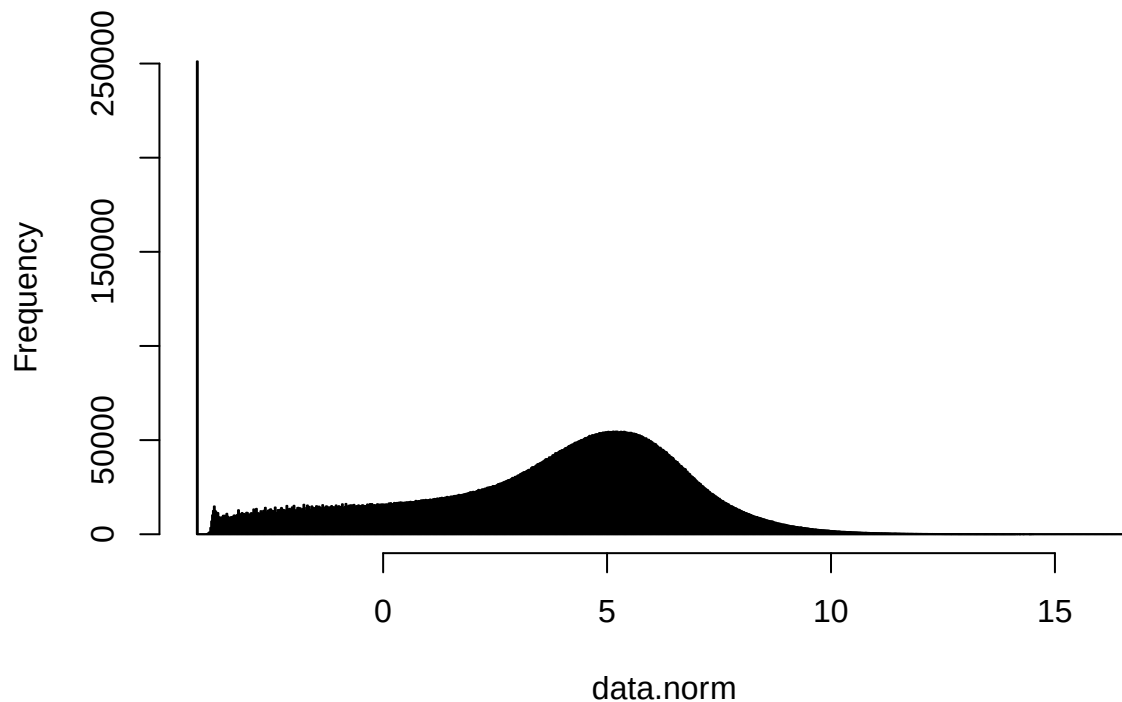
# Sacamos una lista de los genes significativos
listGenes <- rownames(sig)
# Normalizamos los datos para tenerlos en una distribucion continua
data.norm <- cpm(y, log=TRUE, prior.count = 3)

# Observar dataset
cat("Dimensiones del dataset normalizado (genes x muestras): ", dim(data.norm))

## Dimensiones del dataset normalizado (genes x muestras): 16805 981

# Hacemos un histograma con la expresion de los genes normalizados
hist(data.norm, 1000, main = paste("Histograma de la matriz de conteos normalizada "))
```

Histograma de la matriz de conteos normalizada



```
# Guardamos las tablas
write.table(data.norm, "./outputs/01_data_norm.tsv", col.names = T,
            row.names = T, sep = "\t", quote = F)
write.table(listGenes, "./outputs/01_listGenes.tsv", col.names = F,
            row.names = F, sep = "\t", quote = F)
```

Así, ya tenemos la matriz de expresión correctamente normalizada (*data.norm*) para empezar el análisis mecanístico de la activación diferencial.

Exploración de los datos obtenidos

A continuación, se hará un control de calidad mediante una pequeña exploración de la matriz de expresión cruda y normalizada, para lo cual, se utilizará una submuestra de 50 genes.

```
# Obtenemos la matriz de conteos sin normalizar
data <- cpm(exp_mat, log = TRUE, prior.count = 3)
```

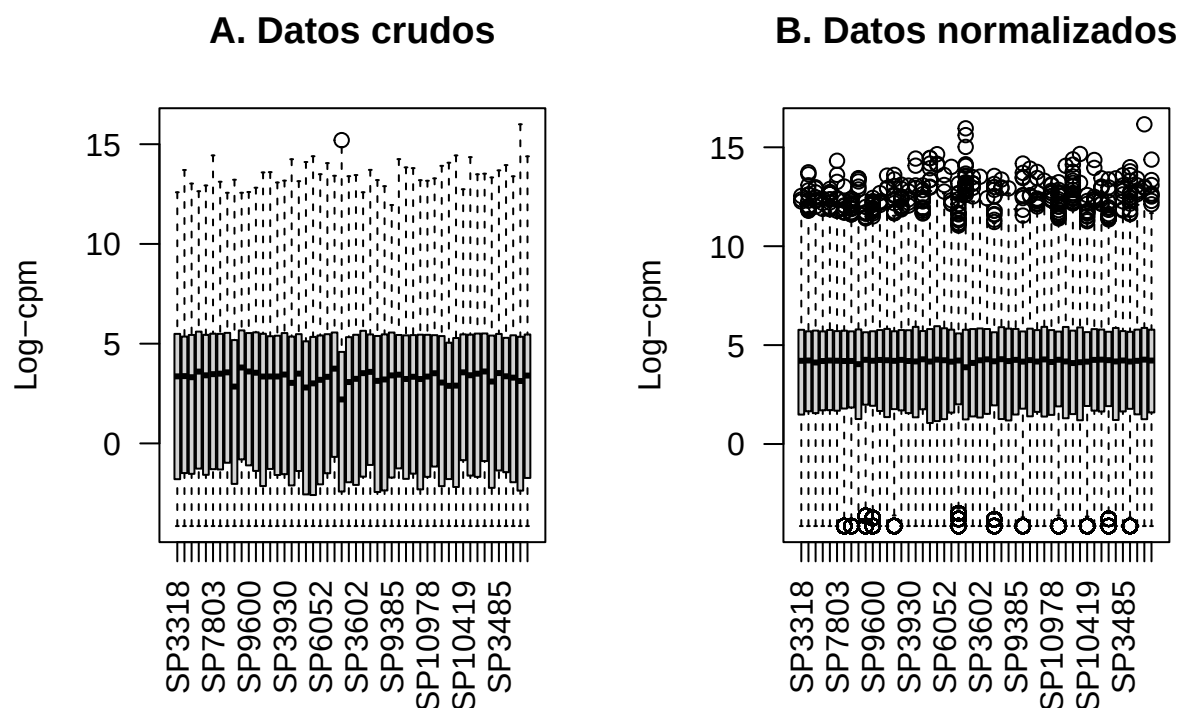
```
# Tomar los índices de las 50 muestras elegidas al azar
set.seed(246)
samples_indices = sample(1:ncol(y), 50)
samples_indices
```

```
## [1] 179 355 445 717 792 424 117 890 706 280 639 961 443 632 368    1 919 840 447
## [20] 296 172 838 850 790 316  88 910 426 821 931 781 330 751 916 531 911 835 937
## [39] 214 477 419 328 423  53 476 206 430 394    6 861
```

```
subset_norm = data.norm[, samples_indices]
subset = data[, samples_indices]
```

```
# Boxplot de datos no normalizados vs. normalizados
#png(file.path(OUT_DIR, '01_boxplots.png'), width=1400, height=700)
par(mfrow=c(1,2), oma = c(0, 0, 2, 0)) # oma añade margen externo
boxplot(subset, las=2, main="")
title(main="A. Datos crudos",ylab="Log-cpm")
boxplot(subset_norm, las=2, main="")
title(main="B. Datos normalizados",ylab="Log-cpm")
mtext("Efecto de la normalizacion TMM sobre 50 muestras aleatorias",
      side = 3, line = 0, outer = TRUE, cex = 1.3, font = 2) # Añade titulo principal
```

Efecto de la normalizacion TMM sobre 50 muestras aleatorias



```
par(mfrow=c(1,1))
```

Implementación del método Hipathia

Hay dos formas principales de implementar el método para calcular la activación diferencial de los genes: mediante los circuitos efectores, como ya brevemente se explicó anteriormente; o colapsando las funciones de activación obtenidas de cada muestra a lo largo del análisis, lo cual está más cercano conceptualmente a un análisis de expresión. Se usará el primer método, utilizando la activación de las vías metabólicas (camino), ya que aporta una mayor granularidad, es decir, una visión más fina de los procesos que están realmente desregulados. Posteriormente, se vuelven a anotar los valores de activación y se le dan esas funciones a cada uno de los nodos efectores.

La herramienta de Hipathia es útil para un amplio rango de campos, como la predicción de clases, el efecto de perturbación (fármacos) sobre las rutas o incluso para la interpretación de variantes. Sin embargo, nos vamos a centrar únicamente en el caso que nos corresponde: el análisis de activación diferencial.

```

# Previamente, instalamos hipathia desde el repo de GitHub para evitar problemas con load_pathways()
suppressPackageStartupMessages({
  library(hipathia) # carga dependencias (SummarizedExperiment, etc.)
})
# Para convertir los IDs
library(AnnotationHub)

set.seed(246) # Semilla de reproducibilidad, para evitar que el
# componente de aleatoriedad cambie los resultados
options(stringsAsFactors = FALSE) # Cambiar los parámetros con los que vamos a trabajar

```

Exploración y control de calidad

Antes de proceder, vamos a ver cómo es la distribución de los genes antes de normalizarlos, para posteriormente comprobar si la normalización funciona debidamente.

```

# Convertir la matriz de expresion en un objeto matriz
exp_matrix <- as.matrix(data.norm)
mode(exp_matrix) <- "numeric"

# Construir una matriz de diseño con las muestras y su grupo
design2 <- data.frame(clinical$sample, ER)
colnames(design2) <- gsub("clinical.sample", "sample",
                        colnames(design2), fixed = T)
colnames(design2)

## [1] "sample" "ER"

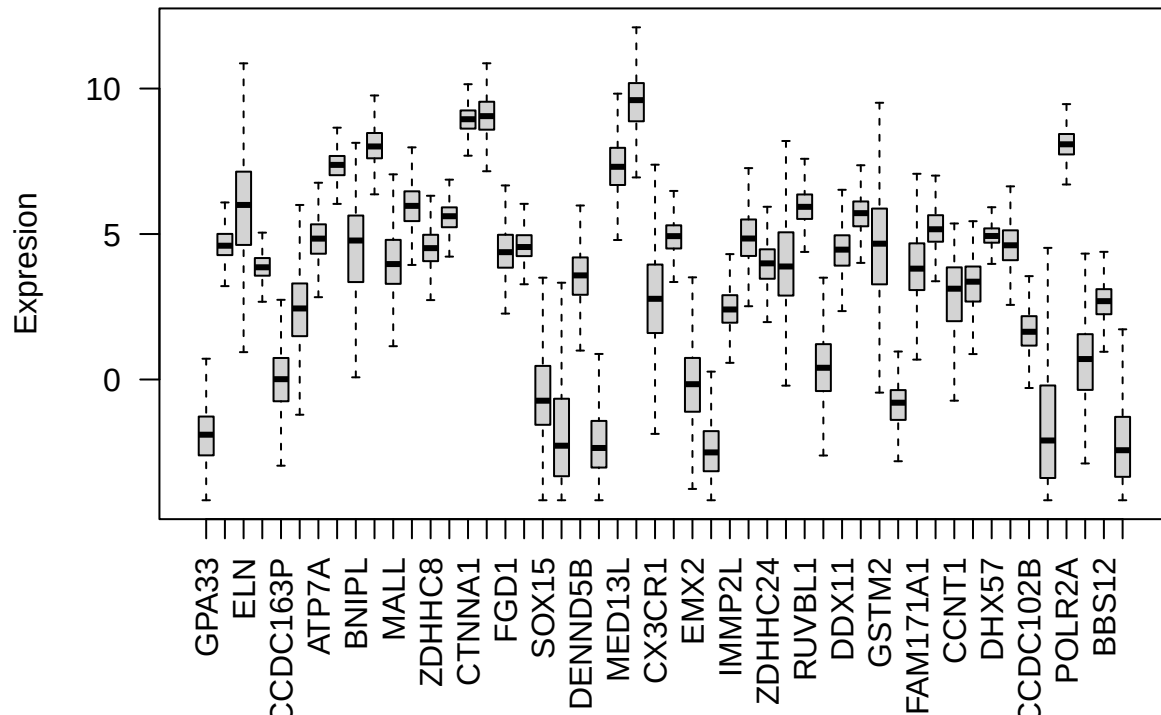
dim(design2)

## [1] 981  2

# Crear subset de genes para graficar el boxplot
set.seed(246)
sel_genes <- sample(seq_len(nrow(exp_matrix)), 50)
boxplot(t(exp_matrix[sel_genes, ]), las=2, main='Distribucion sin normalizar - 50 genes',
        ylab='Expresion', outline=FALSE)

```

Distribucion sin normalizar – 50 genes



Preprocesado para Hipathia: escalado y normalizado

Antes de empezar con el análisis, debemos hacer ciertas modificaciones a los datos requeridas por la herramienta. Para empezar, Hipathia requiere los entrezIDs en los nombres de las filas, para lo que necesitamos acceso a los recursos de anotación en **AnnotationHub**. Esto permitirá acceder a la base de datos KEGG para traducir el nombre del gen o ENSEMBLEID en entrezID. Además, también se necesitará escalar (normalizar) los valores de expresión entre 0 y 1.

Por otro lado, es importante convertir los datos de la matriz de expresión junto con las muestras y el estatus de estrógenos en un objeto de Bioconductor **Summarized Experiment**, el cual contiene la información de manera estructurada.

```
# Construir un objeto SummarizedExperiment
se_raw <- SummarizedExperiment(
  assays = S4Vectors::SimpleList(row = exp_matrix),
  colData = design2
)
str(se_raw)
```

```
## Formal class 'SummarizedExperiment' [package "SummarizedExperiment"] with 5 slots
##   ..@ colData      :Formal class 'DFrame' [package "S4Vectors"] with 6 slots
##   .. ..@ rownames   : chr [1:981] "SP3930" "SP3918" "SP4007" "SP4019" ...
##   .. ..@ nrow       : int 981
##   .. ..@ elementType : chr "ANY"
##   .. ..@ elementMetadata: NULL
##   .. ..@ metadata    : list()
##   .. ..@ listData     :List of 2
```

```

## ..$ sample: chr [1:981] "SP3930" "SP3918" "SP4007" "SP4019" ...
## ..$ ER : Factor w/ 2 levels "Positive","Negative": 1 1 2 1 2 1 1 1 1 ...
## ..@ assays :Formal class 'SimpleAssays' [package "SummarizedExperiment"] with 1 slot
## ..@ data:Formal class 'SimpleList' [package "S4Vectors"] with 4 slots
## ..@ listData :List of 1
## ..$ raw: num [1:16805, 1:981] 2.52 5.48 1.54 3.64 2.83 ...
## ..- attr(*, "dimnames")=List of 2
## ..$ : chr [1:16805] "ALPL" "ALPK1" "ALPK2" "ALPK3" ...
## ..$ : chr [1:981] "SP3930" "SP3918" "SP4007" "SP4019" ...
## ..@ elementType : chr "ANY"
## ..@ elementMetadata: NULL
## ..@ metadata : list()
## ..@ NAMES : chr [1:16805] "ALPL" "ALPK1" "ALPK2" "ALPK3" ...
## ..@ elementMetadata:Formal class 'DFrame' [package "S4Vectors"] with 6 slots
## ..@ rownames : NULL
## ..@ nrows : int 16805
## ..@ elementType : chr "ANY"
## ..@ elementMetadata: NULL
## ..@ metadata : list()
## ..@ listData : Named list()
## ..@ metadata : list()

save(se_raw, file = file.path(OUT_DIR, "02_SumExp_ER"))

# Traducir IDs
se_entrez <- translate_data(se_raw, species = "hsa")

## translated ids = 14856 (0.88)
## untranslated ids = 1949 (0.12)
## multihit ids = 24 (0.0014)

# Normalizar a escala [0,1]
# Escalado global (by_gene=F) / by_quantiles=T puede estabilizar la distribucion
se_norm <- normalize_data(se_entrez, by_quantiles = TRUE, by_gene = FALSE)
str(se_norm)

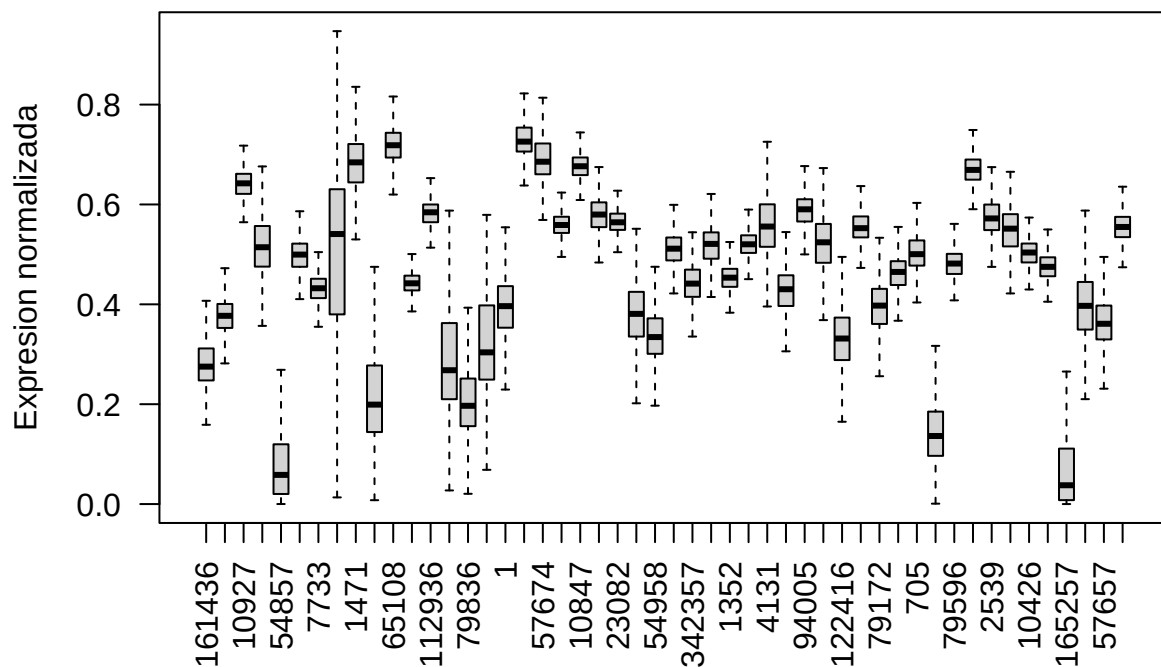
## Formal class 'SummarizedExperiment' [package "SummarizedExperiment"] with 5 slots
## ..@ colData :Formal class 'DFrame' [package "S4Vectors"] with 6 slots
## ..@ rownames : chr [1:981] "SP3930" "SP3918" "SP4007" "SP4019" ...
## ..@ nrows : int 981
## ..@ elementType : chr "ANY"
## ..@ elementMetadata: NULL
## ..@ metadata : list()
## ..@ listData :List of 2
## ..$ sample: chr [1:981] "SP3930" "SP3918" "SP4007" "SP4019" ...
## ..$ ER : Factor w/ 2 levels "Positive","Negative": 1 1 2 1 2 1 1 1 1 ...
## ..@ assays :Formal class 'SimpleAssays' [package "SummarizedExperiment"] with 1 slot
## ..@ data:Formal class 'SimpleList' [package "S4Vectors"] with 4 slots
## ..@ listData :List of 1
## ..$ norm: num [1:14844, 1:981] 0.364 0.76 0.69 0.218 0.714 ...
## ..- attr(*, "dimnames")=List of 2
## ..$ : chr [1:14844] "1" "2" "9" "10" ...
## ..$ : chr [1:981] "SP3930" "SP3918" "SP4007" "SP4019" ...
## ..@ elementType : chr "ANY"
## ..@ elementMetadata: NULL

```

```
## ..@ metadata      : list()
## ..@ NAMES         : chr [1:14844] "1" "2" "9" "10" ...
## ..@ elementMetadata:Formal class 'DFrame' [package "S4Vectors"] with 6 slots
## ..@ rownames      : NULL
## ..@ nrows         : int 14844
## ..@ elementType   : chr "ANY"
## ..@ elementMetadata: NULL
## ..@ metadata      : list()
## ..@ listData       : Named list()
## ..@ metadata       : list()

## Boxplot tras normalización ##
# Matriz de expresion normalizada con los entrezIDs para grafico
norm_mat <- assay(se_norm)
# Definir el subset con los mismos genes que anteriormente
set.seed(246)
sel_genes2 <- sample(seq_len(nrow(norm_mat)), 50)
boxplot(t(norm_mat[sel_genes2, ]), las=2, main='Distribucion normalizada [0-1] - 50 genes',
        ylab='Expresion normalizada', outline=FALSE)
```

Distribucion normalizada [0-1] - 50 genes



Cálculo de activación de circuitos

A continuación, vamos a cargar **todas las rutas** de referencia disponibles para el *Homo sapiens* en KEGG (por ello necesitábamos el entrezID). A posteriori, se creará una matriz con los valores computados de activación que se obtuvieron mediante la función *hipathia*.

Problema: Las versiones de *hipathia* posteriores a la 3.5 dan un error con el recurso “AH69111” cuando

se utiliza la función `load_pathways()`, por lo que no es posible descargar las rutas de KEGG. Por lo tanto, debemos descargar la última versión que no da problemas, y para evitar problemas de compatibilidad con otros paquetes, reduciremos la versión de todo la librería de Bioconductor a la 3.18, pero aquí tenemos otro problema: la versión actual de R, la 3.5.2, no soporta una versión tan antigua de la librería, por lo que también tendremos que trabajar con una versión más antigua de R.

Esto es más complicado, pues si lo hiciéramos a lo bruto, tendríamos que desinstalar R y volverlo a instalar con la versión deseada. Para evitarnos esto, haremos uso de **Docker**, de manera que podamos usar la versión 3.18 de Bioconductor dentro de la versión 4.3 de R sin alterar la versión instalada en este ordenador.

```
### Esta celda esta siendo ejecutada desde Docker
# bioconductor/bioconductor_docker:RELEASE_3_18
library(hipathia)

# Cargar rutas de KEGG
#pathways <- load_pathways(species = "hsa")

# Ejecutar Hipathia: devuelve un objeto con nodos efectores y circuitos
#res <- hipathia(se_norm, pathways)

# Extraer la matriz de activacion de los circuitos
#path_mat <- get_paths_data(res, matrix = TRUE)
# Guardar objetos para que sean reutilizables
#saveRDS(pathways, file = file.path(OUT_DIR, '03_pathways_object.rds'))
#saveRDS(res, file = file.path(OUT_DIR, '03_results_object.rds'))
#saveRDS(path_mat, file = file.path(OUT_DIR, '03_path_activity_matrix.rds'))
```

Se obtienen **146 rutas metabólicas** de KEGG, cuya última actualización es del 2023-10-23. Entonces, ahora que por fin tenemos los circuitos de **todas las rutas** (no sólo las fisiológicas), volvemos a trabajar en local con RStudio 4.5.2. Cargamos las rutas que descargamos en Docker con `load_pathways`

```
pathways <- readRDS("D:/Académico/Universitario/Master en Bioinformática/Cursos/Módulo 4. Metodología de
path_mat <- readRDS("D:/Académico/Universitario/Master en Bioinformática/Cursos/Módulo 4. Metodología de
res <- readRDS("D:/Académico/Universitario/Master en Bioinformática/Cursos/Módulo 4. Metodología de la
dim(path_mat)
```

```
## [1] 1876 981
```

Análisis de activación diferencial: Positive vs Negative

Ahora sí que podemos empezar finalmente con el análisis mecanístico con Hipathia, ya que tenemos la matriz de expresión normalizada entre 0 y 1, con los nombres de los genes con su entrezID y las rutas metabólicas de la base de datos KEGG guardados en la matriz de activación.

Entonces, vamos a estudiar los valores de activación de los circuitos de los genes que derivan en determinadas funciones para las muestras luminales de cáncer de mama. Así, usaremos el test estadístico *Wilcoxon* para quedarnos con los circuitos significativos, y a continuación, se utilizará un *método no paramétrico* para calcular los caminos elementales de forma independiente, computándolos como un todo. Obteniendo así resultados más robustos.

```
# Convertir la matriz de diseño en un vector
ERvec <- as.character(design2$ER)
names(ERvec) <- design2$sample
head(ERvec, 5)
```

```
##      SP3930      SP3918      SP4007      SP4019      SP3995
## "Positive" "Positive" "Negative" "Positive" "Negative"
```

```

table(ERvec)

## ERvec
## Negative Positive
##      224      757

# Wilcoxon por circuito (no parametrico)
# Ajustar los datos por multiples comparaciones
comp_paths <- do_wilcoxon(path_mat, ERvec[colnames(path_mat)], g1='Positive', g2='Negative',
                          paired=FALSE, adjust=TRUE, order=TRUE)

write.csv(comp_paths, file = file.path(OUT_DIR, '04_wilcoxon_circuitos.csv'))

# Seleccionar circuitos significativos (FDR < 0.05 si existe esa columna)
fdr_col <- if ('FDRp.value' %in% colnames(comp_paths)) 'FDRp.value' else 'p.value'
sig_comp <- comp_paths[comp_paths[[fdr_col]] < 0.05, , drop=FALSE]
# Traducir a nombres legibles "nombre de ruta hsa: nombre gen efector"
sig_comp$path_name <- get_path_names(pathways, rownames(sig_comp))
write.csv(sig_comp, file = file.path(OUT_DIR, '04_wilcoxon_circuitos_significativos.csv'))

cat("Hay", nrow(sig_comp), "circuitos significativos (", fdr_col, " < 0.05). ")

## Hay 1565 circuitos significativos ( FDRp.value < 0.05).

cat("De los cuales,", table(sig_comp$`UP/DOWN`)[1], "son DOWNREGULADOS y",
    table(sig_comp$`UP/DOWN`)[2], "son UPREGULADOS")

```

De los cuales, 836 son DOWNREGULADOS y 729 son UPREGULADOS

Se obtienen bastantes circuito significativos, pertenecientes a una gran variedad de rutas de señalización diferentes. Vamos a ver los 10 circuitos más significativos a cual pertenecen.

```

# Rutas de señalizacion de los 10 circuitos mas significativos
sig_comp[1:10, ]

```

##	UP/DOWN	statistic	p.value	FDRp.value	
## P-hsa04961-26	UP	17.27944	1.879087e-112	1.762584e-109	
## P-hsa04961-17	UP	17.27944	1.879087e-112	1.762584e-109	
## P-hsa04919-39	UP	20.74875	1.261821e-95	7.890588e-93	
## P-hsa04915-58	UP	20.10474	6.725914e-90	3.154454e-87	
## P-hsa05205-103	UP	19.73669	1.046731e-86	3.927335e-84	
## P-hsa04917-57	UP	19.65750	5.000444e-86	1.563472e-83	
## P-hsa04915-40	UP	19.43281	4.085362e-84	1.094877e-81	
## P-hsa04930-26	UP	18.08680	4.059230e-73	9.518893e-71	
## P-hsa04930-7	UP	18.07015	5.489307e-73	1.144216e-70	
## P-hsa05010-37	UP	17.86935	2.048371e-71	3.842744e-69	
##					path_name
## P-hsa04961-26	Endocrine and other factor-regulated calcium reabsorption:				TRPV5*
## P-hsa04961-17	Endocrine and other factor-regulated calcium reabsorption:				CALB1
## P-hsa04919-39					Thyroid hormone signaling pathway: ESR1
## P-hsa04915-58					Estrogen signaling pathway: CREB3***
## P-hsa05205-103					Proteoglycans in cancer: CCND1
## P-hsa04917-57					Prolactin signaling pathway: ESR1
## P-hsa04915-40					Estrogen signaling pathway: CREB3**
## P-hsa04930-26					Type II diabetes mellitus: INS*
## P-hsa04930-7					Type II diabetes mellitus: ATP
## P-hsa05010-37					Alzheimer's disease: MAPT

Se observa que, los 10 caminos son upregulados y las rutas de señalización a las que pertenecen son variadas, con muchas de ellas relativas a genes importantes en el cáncer de mama luminal, como rutas del sistema endocrino, de la hormona tiroidea y de los estrógenos,. Además, hay rutas que se alejan más del comportamiento intrínseco de un tumor, así como la diabetes tipo II y, más interesantemente, la ruta relativa a la enfermedad de Alzheimer.

Estas rutas de señalización tan variadas, nos muestran cómo se conectan el metabolismo, las hormonas y la neurodegeneración mediante una **comunicación cruzada** [3].

- **Sistema endocrino.** La activación de las vías de señalización de estrógenos, prolactina y hormona tiroidea convergen en los nodos **ESR1** y **CREB3**, lo que indica una respuesta hormonal fuertemente coordinada que actúa como integradora sistémica. La hiperactivación de la reabsorción del calcio (dependiente de TRPV5 y CALB1) refuerza esta idea.
- **Estrés celular y proliferación.** La aparición del circuito “proteoglicanos en cáncer” mediado por **CCND1** sugiere que las células están recibiendo potentes señales de progresión del ciclo celular, que unido a la fuerte presencia de CREB3, indica un estado de altísima demanda metabólica y estrés celular.
- **Conexión metabólico-neurodegenerativa.** Se observa una activación simultánea de las vías de la **Diabetes Mellitus Tipo II** (circuitos INS y ATP) y de la **Enfermedad de Alzheimer** (circuito MAPT, codificante de la proteína Tau). Resultados más recientes demuestran que los estrógenos tienen un papel neuroprotector frente a la resistencia a la insulina, y su alteración es un factor de riesgo crítico para el desarrollo de Alzheimer, una patología que la literatura reciente apoda como “diabetes tipo 3” debido a la dependencia cruzada de estas vías [4].

A continuación, es imprescindible visualizar los resultados para obtener una discusión más significativa y robusta, para lo cual se utilizarán los gráficos de PCA y heatmap, representando los circuitos más relevantes, según su p-valor ajustado (FDR). Sin embargo, antes de hacer el PCA, eliminaremos los circuitos que tienen varianza cero, pues sólo aportan ruido sin aportar ninguna información diferencial.

```
# Calculamos la varianza de cada circuito (filas en tu path_mat original)
varianzas <- apply(path_mat, 1, var)

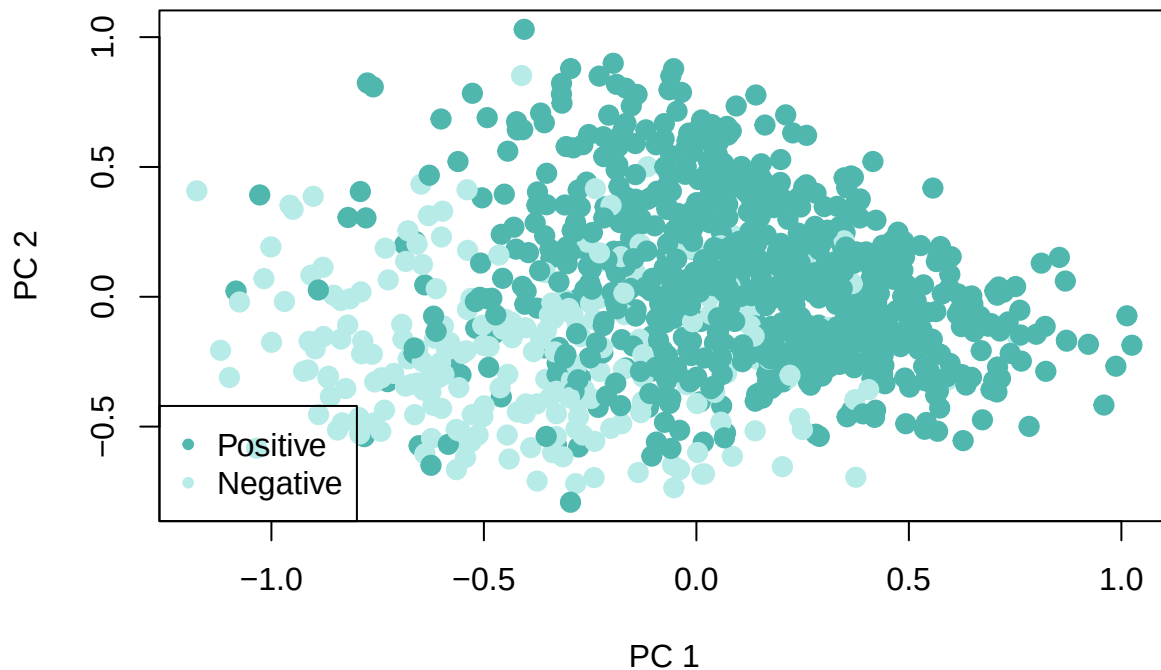
# Filtramos la matriz para conservar SOLO los circuitos que varían (varianza > 0)
path_mat_limpia <- path_mat[varianzas > 0, ]
dim(path_mat_limpia)

## [1] 1866 981

# Calculamos el PCA creando un objeto princomp
pca_model <- do_pca(path_mat_limpia[seq_len(ncol(path_mat_limpia)),])

# Hacemos la grafica usando la columna "scores" que se creo en pca_model
pca_plot(pca_model,
  group = ERvec[rownames(pca_model$scores)],
  main='PCA (actividad de circuitos) Positive vs Negative')
```

PCA (actividad de circuitos) Positive vs Negative



A pesar de que la diferenciación entre ER+ y ER- no es perfecta, hay una relevante acumulación de la actividad de los circuitos para cada clase, lo cual implica que puedan ser clasificados mediante modelos lineales con relativa precisión.

A continuación, vamos a visualizar el heatmap, lo cual complementa muy bien al PCA y al posterior informe interactivo de los mapas de señalización.

```
# Heatmap completo de los top circuitos por FDR/pvalue
library(pheatmap)

# Ordenar tabla de resultados por la columna del FDR (de menor a mayor)
comp_paths_sorted <- comp_paths[order(comp_paths$FDRp.value), ]
# Extraer el top 20
n_top <- 20
sel_fdr <- head(rownames(comp_paths_sorted), n_top)
# Nos aseguramos de cruzar solo los que existen en la matriz
sel_fdr_validos <- sel_fdr[sel_fdr %in% rownames(path_mat)]

# Preparar la anotacion clinica
# pheatmap necesita que la informacion de los pacientes sea un data.frame
# y que los nombres de las filas coincidan EXACTAMENTE con las columnas de la matriz
annotation_df <- data.frame(Fenotipo = ERvec)
rownames(annotation_df) <- colnames(path_mat)

# Personalizar los colores de la leyenda clinica
ann_colors <- list(
  Fenotipo = c("Positive" = "#008080", "Negative" = "#87CEEB")
)
```

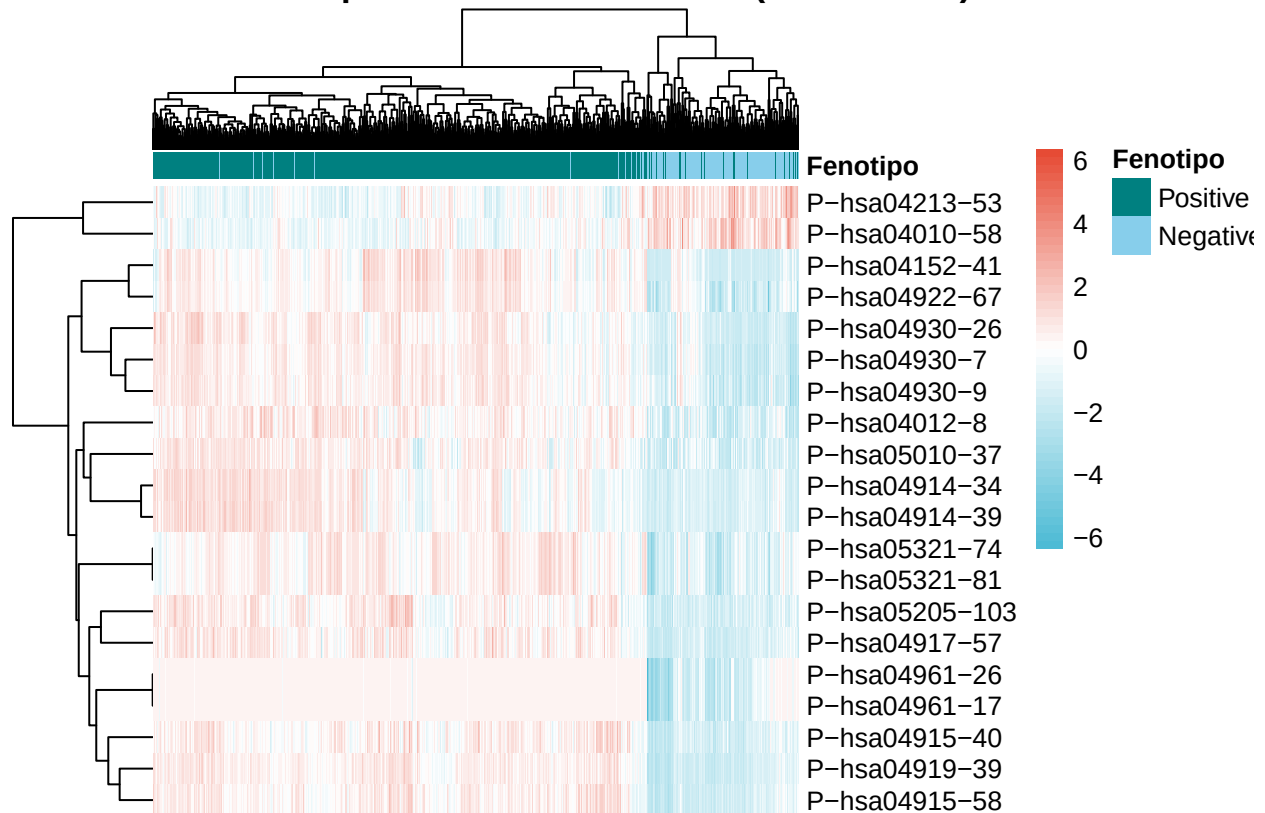
```

)

# Renderizar el heatmap definitivo
pheatmap(
  mat = path_mat[sel_fdr_validos, ], # Matriz filtrada con los top 20 circuitos
  annotation_col = annotation_df, # Barra de pacientes (ER+ vs ER-)
  annotation_colors = ann_colors,
  scale = "row", # Convierte los valores a Z-score para máximo contraste visual
  color = colorRampPalette(c("#4DBBD5", "white", "#E64B35"))(50),
  cluster_rows = TRUE, # Agrupa los circuitos biológicamente similares
  cluster_cols = TRUE, # Agrupa a los pacientes "a ciegas" (tu validación)
  show_colnames = FALSE, # Ocultamos los IDs de los pacientes para que quede limpio
  fontsize_row = 10,
  main = "Firma Mecanística: Top 20 Circuitos Activados (ER- vs ER+)"
)

```

Firma Mecanística: Top 20 Circuitos Activados (ER- vs ER+)



Los **tonos cálidos** representan los circuitos que están **sobreactivados**, mientras que los **tonos fríos** representan los circuitos **inhibidos**.

Obtenemos unos resultados excelentes, lo cual se puede ver en cómo el algoritmo, el cual es de aprendizaje no supervisado, ha clasificado a los pacientes en función de, únicamente la activación de 20 circuitos. Esto queda representado en la parte superior, donde el dendrograma clasifica las muestras mediante los valores de activación de los circuitos y la barra con colores verde y azul, muestra el vector de clínico de muestras *ERvec*. La gran mayoría de muestras han sido correctamente clasificadas, donde la rama izquierda del dendrograma (árbol) representa las muestras de ER+ y la rama derecha, las muestras ER-. Esto es muy potente pues se ha encontrado una **firma biológica que define el fenotipo**.

Si nos fijamos en los valores de activación de las muestras ER+ (bloque de la izquierda), la mayoría de circuitos están en la media de activación (tonos cálidos no muy intensos) o muy poco activados (tonos blancos), mientras que los mismos circuitos de las muestras ER- están muy reprimidos (tonos azules más intensos). Esto indica que el algoritmo ha detectado que **la vía de señalización de estrógenos está encendida en los ER+ y completamente apagada en los ER-**, lo cual sugiere que el modelo funciona muy bien.

Estos resultados serán complementados de manera estupenda en el posterior informe interactivo donde se verán los mapas de señalización.

Informe interactivo

A continuación, se obtendrá el informe interactivo que la herramienta de Hipathia ofrece, lo cual es el broche perfecto para un análisis completo y robusto, técnica y biológicamente.

```
# Diferencia nodos
colors.de <- node_color_per_de(res, pathways, ER, "Positive","Negative")

# Creamos y visualizamos el report, debemos abrir un navegador y pegar la
# url que nos indique la función visualize_report()
url<-create_report(comp_paths, pathways, "NEG_vs_POS", node_colors = colors.de)

## Creating report folders...
## Creating pathways folder...
## Creating HTML index...
visualize_report(url, port=4054) #El número de puerto podemos ir cambiándolo

## Serving the directory C:\Users\javia\AppData\Local\Temp\RtmpuEzhIg\NEG_vs_POS\pathway-viewer at http
## Open a web browser and go to URL http://127.0.0.1:4054
#servr::daemon_stop(1) #para cerrar el puerto y poderlo usar de nuevo, comentado para knitr

message('FIN. Copia y pega la url proporcionada en un navegador para visualizar las rutas \n
        y los circuitos diferencialmente activados de manera interactiva.\n
        Importante cerrar la conexión al servidor antes de abrir otro puerto.')

## FIN. Copia y pega la url proporcionada en un navegador para visualizar las rutas
##
##         y los circuitos diferencialmente activados de manera interactiva.
##
##         Importante cerrar la conexión al servidor antes de abrir otro puerto.
```

Accedemos al enlace proporcionado por Hipathia y entramos en una web con un servidor privado que nos muestra las 146 rutas de señalización. Nos vamos a centrar en aquellas más importantes y representativas de los circuitos de activación del cáncer de mama.

Tal y como se vio en la sección del análisis diferencial, hay una conexión cruzada entre 3 sistemas biológicos que se comunican mediante las rutas de señalización: **el motor hormonal, el secuestro metabólico y la consecuencia iónica.**

En ambos circuitos, se puede ver como cascadas enteras (con flejas rojas) convergen hacia **ESR1** y **CREB3**. Todos los caminos desde los receptores de membrana hasta el núcleo está hiperactivada en el fenotipo *Negative*. Más interesantemente, en la ruta de estrógenos, se observa que las señales derivan hacia CREB3, etiquetado como *Unfolded protein response*, lo que indica a nivel celular que esta hiperactivación está generando un alto estrés en el retículo endoplasmático, típico de las células con una altísima tasa de proliferación.

El mapa de la Diabetes tipo II es un escándalo visual, hay una sobreexcitación enorme desde los receptores INSR pasando por el nodo clave **IRS1** y activando las quinasas asociadas (MAPK, PI3K). Al mismo tiempo,

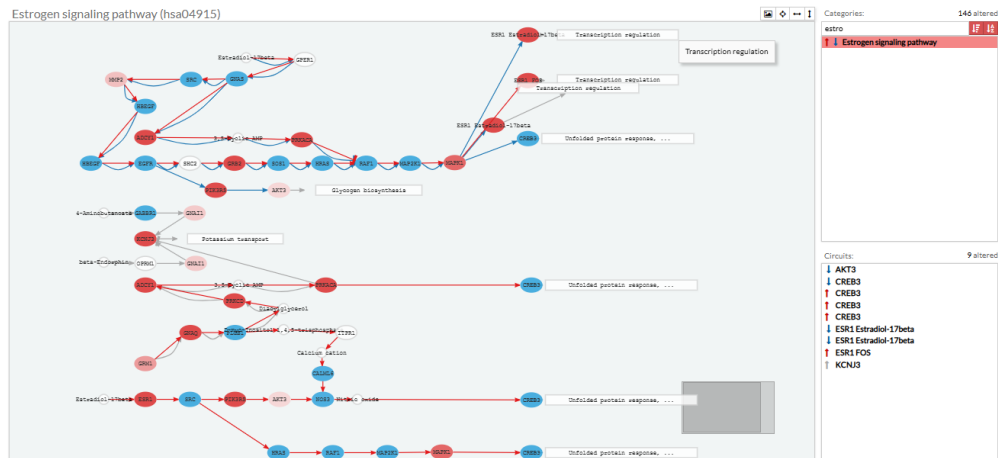


Figure 1: Mapa de la ruta de señalización de estrógenos

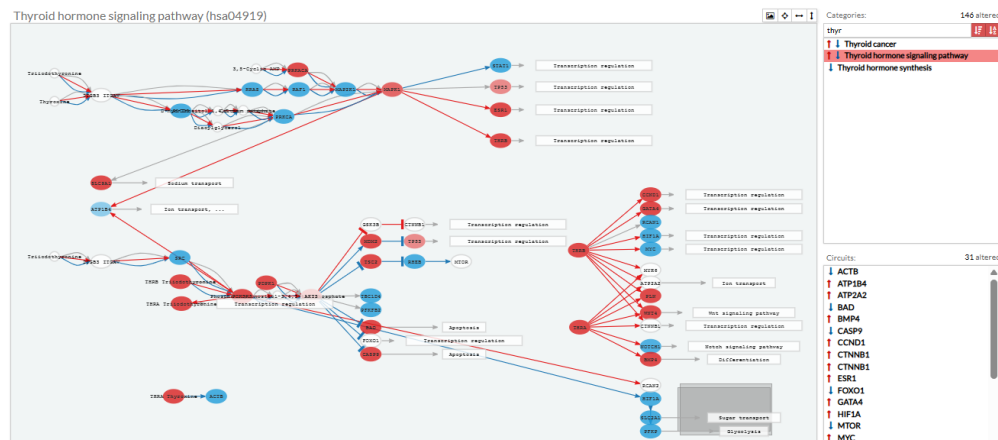


Figure 2: Mapa de la ruta de señalización de tiroides

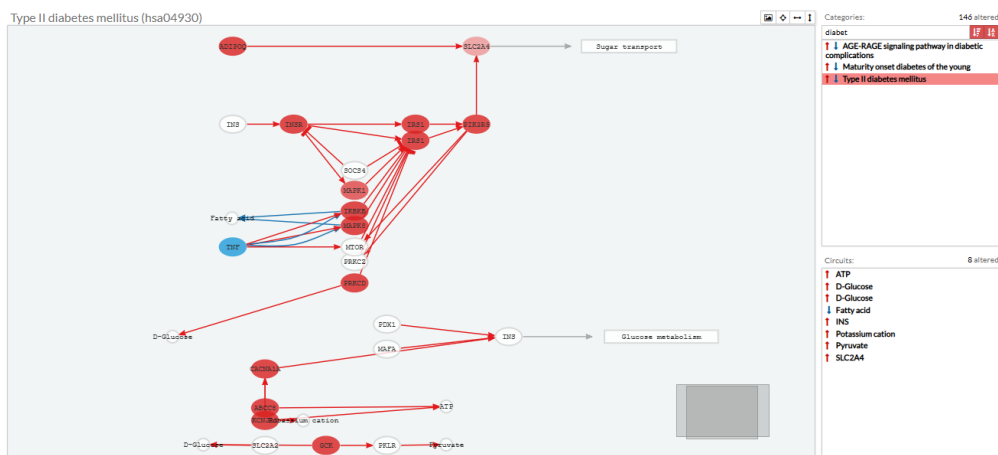


Figure 3: Mapa de la ruta de señalización de diabetes tipo II

se observa es la comunicación cruzada metabólica, ya que las células del fenotipo *Negative* no solo tienen alteraciones hormonales, sino que han reprogramado por completo su sensibilidad a la insulina y su maquinaria bioenergética para asegurar el flujo constante de glucosa (SLC2A4) y **ATP**. Están energéticamente aceleradas a fondo.

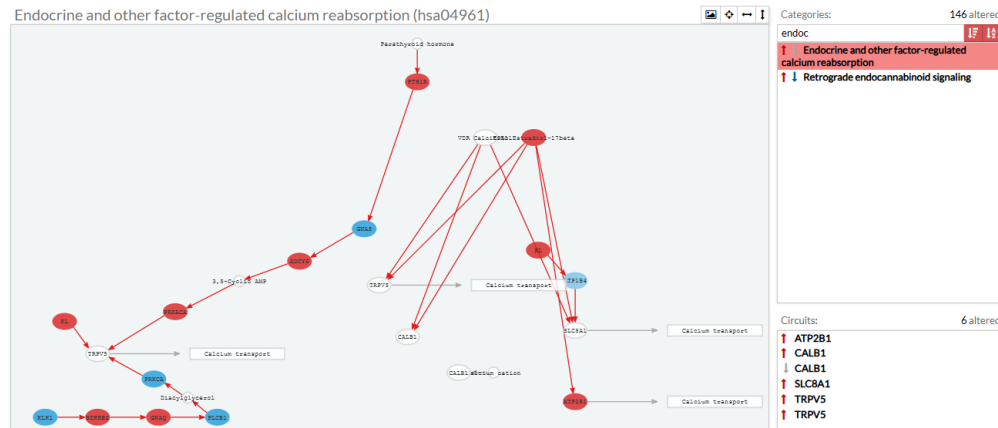


Figure 4: Mapa de la ruta de señalización de endocrinos

Este mapa de reabsorción de calcio cierra el círculo. Vemos cómo las señales de la vitamina D y el estradiol impulsan directamente a **TRPV5** y **CALB1**. El calcio es un segundo mensajero celular crítico, de manera que su hiperactivación coordinada aquí sugiere que la homeostasis iónica está alterada para favorecer procesos dinámicos como la motilidad celular o la secreción.

Todo esto queda respaldado matemáticamente gracias al gráfico *heatmap*. La limpieza con la que se separan los clústeres nos confirma que esta firma de **hiperactivación metabólico-endocrina** es el rasgo definitivo y absoluto que **separa a los pacientes de receptor de estrógenos positivo y negativo**. Es decir, se trata de una red donde las hormonas encienden los motores del metabolismo y reprograman la célula.

Bibliografía

- [1] Hidalgo MR, Cubuk C, Amadoz A, Salavert F, Carbonell-Caballero J, Dopazo J. High throughput estimation of functional cell activities reveals disease mechanisms and predicts relevant clinical outcomes. *Oncotarget*. 2017
- [2] TFM: Elena Rodríguez de Tiedra, DESARROLLO DE UN MÓDULO DE ANÁLISIS TRANSCRIP-TÓMICO, MECANÍSTICO E INTEGRATIVO. Universidad Europea.
- [3] Diabetes & Alzheimer’s – save your brain by going to the gym | Ewan McNay | TEDxAlbany. <https://www.youtube.com/watch?v=7cVWh7P5QZM&start=1>
- [4] Kroner, Z. (2009). The relationship between Alzheimer’s disease and diabetes: Type 3 diabetes?. *Alternative Medicine Review*, 14(4), 373.

Declaración de uso de IA

La Inteligencia Artificial se ha utilizado como soporte a lo largo de toda la actividad, tanto para comprender procesos técnicos del código y redactar el mismo, como en la discusión y análisis de algunos resultados, especialmente en la discusión de los mapas de rutas y del heatmap.

```
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31 ucrt)
```

```

## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=English_United Kingdom.utf8
## [2] LC_CTYPE=English_United Kingdom.utf8
## [3] LC_MONETARY=English_United Kingdom.utf8
## [4] LC_NUMERIC=C
## [5] LC_TIME=English_United Kingdom.utf8
##
## time zone: Europe/Madrid
## tzcode source: internal
##
## attached base packages:
## [1] stats4      stats      graphics  grDevices  utils      datasets  methods
## [8] base
##
## other attached packages:
## [1] pheatmap_1.0.13      hipathia_2.13.1
## [3] MultiAssayExperiment_1.34.0 SummarizedExperiment_1.38.1
## [5] Biobase_2.68.0       GenomicRanges_1.60.0
## [7] GenomeInfoDb_1.44.3  IRanges_2.42.0
## [9] S4Vectors_0.46.0     MatrixGenerics_1.20.0
## [11] matrixStats_1.5.0    AnnotationHub_3.16.1
## [13] BiocFileCache_3.0.0  dbplyr_2.5.1
## [15] BiocGenerics_0.56.0  generics_0.1.4
## [17] igraph_2.2.1         edgeR_4.6.3
## [19] limma_3.64.3
##
## loaded via a namespace (and not attached):
## [1] tidyselect_1.2.1      dplyr_1.2.0           farver_2.1.2
## [4] blob_1.3.0           filelock_1.0.3        Biostrings_2.76.0
## [7] fastmap_1.2.0         promises_1.5.0        digest_0.6.39
## [10] mime_0.13            lifecycle_1.0.5       statmod_1.5.1
## [13] KEGGREST_1.48.1      RSQlite_2.4.5         magrittr_2.0.4
## [16] compiler_4.5.2       rlang_1.1.7          tools_4.5.2
## [19] yaml_2.3.12          knitr_1.51           S4Arrays_1.8.1
## [22] bit_4.6.0            curl_7.0.0           DelayedArray_0.34.1
## [25] RColorBrewer_1.1-3   abind_1.4-8          withr_3.0.2
## [28] purrr_1.2.1          grid_4.5.2           preprocessCore_1.70.0
## [31] scales_1.4.0         cli_3.6.5            rmarkdown_2.30
## [34] crayon_1.5.3         otel_0.2.0           rstudioapi_0.18.0
## [37] httr_1.4.7           BiocBaseUtils_1.10.0 DBI_1.2.3
## [40] cachem_1.1.0         AnnotationDbi_1.70.0 BiocManager_1.30.27
## [43] XVector_0.48.0       vctrs_0.7.1          Matrix_1.7-4
## [46] jsonlite_2.0.0       bit64_4.6.0-1        locfit_1.5-9.12
## [49] servr_0.32           glue_1.8.0           gtable_0.3.6
## [52] BiocVersion_3.21.1   later_1.4.5          UCSC.utils_1.4.0
## [55] tibble_3.3.1         pillar_1.11.1        rappdirs_0.3.4
## [58] htmltools_0.5.9      GenomeInfoDbData_1.2.14 R6_2.6.1
## [61] httr2_1.2.2          evaluate_1.0.5       lattice_0.22-7

```

## [64]	png_0.1-8	memoise_2.0.1	httpuv_1.6.16
## [67]	Rcpp_1.1.1	SparseArray_1.8.1	xfun_0.56
## [70]	pkgconfig_2.0.3		